

CS 3210

4.5.1

Let us first try and decode the instruction. Over here, the hex value is 0xF8014062, so its binary will be 11111000000 000010100 00 00011 00010 and hence we can say that its opcode will be STUR(since it is 11111000000), its address which is a 9 bit offset will be 20 (which is decimal value of 000010100), the op2 is 00, Rn is X3 (since it has a value of 00011) and Rt is X2(since its value is 00010), so we can say that the given instruction is STUR X2, [X3, #20].

In this STUR instruction, the “top” sign-extended and “shift left 2” block in figure 4.23 is basically used for branch targets. This unit basically takes bits 25:0 of the instruction as a 26 immediate value. Since the most significant bit of that field is 0 the value is basically sign-extended to a 64-bit positive number. After the sign extension, the shift-left extension, the “shift left 2” unit multiplies that value by 4 basically by shifting it to left by 2 positions. The exact numeric value is not used by STUR because the STUR is not a branch, but its outputs are basically a sign-extended 64-bit version of the 26-bit immediate and this same value is shifted left by 2, which produces a value 4 times larger.

Here, although these values are generated, they are ignored during execution because STUR does not perform a branch.

4.5.2

The ALU control unit receives two inputs: ALUOp from the main control unit and Opcode bits from the instruction. Since this is a store instruction (STUR), the main control here generates an ALUOp of 00, which basically tells the ALU to perform an addition. Then, over here, the opcode is passed into the ALU control unit, but it does not affect the result because ALUOp = 00 forces the ALU to perform addition.

4.5.3

As STUR is basically not a branch of any type, in this case, the processor will just move to the next sequential instruction, then the PC adder basically adds 4 to the current PC to point to the next instruction, and then later on, the next PC selection mux chooses this value because the branch control signal is 0.

4.5.4

1) Reg2Loc multiplexer

Over here, this mux basically chooses which register number must become the second read address for the register file. It then basically chooses between the register in bits 20-16 and the

register in bits 4-0, then for STUR, the control signal basically selects bits 4-0, which corresponds to register X2.

2) ALUSrc multiplexer

In this case, the mux basically decided whether the ALU must use the register value or the immediate value as its second input. Over here, for STUR, the control basically selects the immediate value, which is 20.

3) MemtoReg multiplexer

In this case, the mux normally chooses between an ALU output and a data memory output for writing back to a register, however for STUR, there is no register written over here, so this mux's output basically gets ignored over here.

4) PC-selection multiplexer

This mux essentially selects between the $PC + 4$ and the branch target address. Since STUR is not a branch, the mux selects $PC + 4$.

4.5.5

Over here, for this instruction, the ALU and both adders gave the same inputs, which are:

- ALU inputs: In this case, the first ALU input comes from register X3, which is the base register (Rn). The second one was the sign-extended is the sign-extended 9-bit immediate value, which is 20, chosen by the ALUSrc multiplexer. The ALU basically adds these 2 values to produce the effective memory address for the store.
- PC adder (top adder): This adder always receives the current PC and the constant 4, producing $PC + 4$.
- Branch adder (lower order): Over here, we can basically say that this adder receives $PC+4$ (that is, from the top adder) and the output of the top-sign-extend and shift-left-2 unit. The second input in the branch is immediate shifted left by 2, taken from bits 25:0 of the instruction. Although the branch over here is computed, it is not used for STUR since no branch is taken.

4.5.6

Over here, we can say that the register file basically receives the following:

- 1) Read register 1 = X3, from the instruction bits 9-5
- 2) Read Register 2 = X2, which is selected through the Reg2Loc mux
- 3) Write register = X2, which is from the instruction bits 4-0, but over here writing is disabled
- 4) Write data = irrelevant, since it is ignored because writing is disabled.
- 5) RegWrite control signal = 0, which means “do not write”

4.16.1

Over here we can list the stage latencies as follows:

- 1) IF: 250 ps
- 2) ID: 350 ps
- 3) EX: 150 ps
- 4) MEM: 300 ps
- 5) WB: 200 ps

In non-pipelined, basically 1 instruction needs to go through all 5 stages in a particular clock cycle, hence the clock period must be long enough to fit every stage:

- Total = 250 + 350 + 150 + 300 + 200 = 1250 ps

Hence, for non pipelined clock cycle time is 1250 ps.

In case of a pipeline one, each stage has basically its own clock cycle and over here the clock period must be as long as the slowest stage, which is 350 ps in our case, hence 350 ps is the pipelined clock cycle time as well.

4.16.2

Over here, latency is the basically the time from when the instruction starts until it finishes. So, since in case of a pipelined type, an instruction basically goes through all 5 stages, where each state is a cycle, even though many instructions overlap for a single instruction. It spends 1 cycle in IF, then 1 in ID and then 1 in EX and MEM and 1 in WB, which makes it 5 cycles in total, where each cycle is 350 ps, so the latency becomes 1750 ps (5 x 350).

In case of a non pipelined type, an instruction basically goes all stages in 1 long cycle, so the latency for LDUR is just that 1 cycle, that is, 1250 ps.

4.16.3

In this case, we can choose one stage and essentially split it into two shorter stages, where each has half of its original latency. Our primary objective is to minimize the maximum stage delay, which will enable us to determine the clock period. Our current stage latencies can be listed as follows:

- IF: 250 ps
- ID: 350 ps
- EX: 150 ps
- MEM: 300 ps
- WB: 200 ps

Over here, we can see that the ID stage would be suited the best to split since it is the slowest one. If we are able to split into 2 pieces, then each new ID half will be equal to 175 ps ($350/2$), so now the pipeline stages will look like this:

- IF: 250 ps
- ID1: 175 ps
- ID2: 175 ps
- EX: 150 ps
- MEM: 300 ps
- WB: 200 ps

The new slowest stage is now MEM, which is 300 ps, so the new pipelined clock cycle is also 300 ps.

4.16.4

The data memory is actually only used by LDUR (load) instructions and STUR (store) instructions, and from the mix that has been given to us, we can say that:

- ALU/Logic = 45%
- Jump/Branch = 20%
- LDUR = 20%
- STUR = 15%

Since only LDUR and STUR access the data memory, we can say that 35% [$20\%(\text{LDUR}) + 15\%(\text{STUR})$] is the data memory utilization, since with no stalls and hazards, once the pipeline becomes full, one instruction basically completes every cycle, and one instruction is in MEM every cycle and then fraction of those cycles in which MEM actually does useful work is actually the fraction of load/store instructions, hence the data memory utilization is 35%.

4.16.5

Over here, the write register port in the register file is only used by instructions that may write a result into a register. By looking at the instruction types, ALU/Logic(which writes a result to a register and LDUR (which loads data from a memory to a register) will be the right match for this question since they match the definition which we described at the beginning of this answer. Hence, the register writing instructions are ALU/Logic (45%) and LDUR (20%), totaling 65%. Over here we have a full, hazard-free pipeline and 1 instruction is in WB cycle, but the write port is only used when that instruction writes a register. Hence, the write-register port utilization is 65%.

4.17

Let us think of the pipeline as a factory assembly line with k number of steps. Our first step is to fill the pipeline(which is the startup cost), so the first instruction has to go through all k stages and it takes k cycles before the first instruction finishes(which means one stage per cycle). Then, the steady state follows, which occurs once per cycle. So, over here, once the pipeline is full, each new cycle finishes one more instruction, so to finish the remaining $(n-1)$ instructions, we need $(n-1)$ additional cycles. So, we must add k cycles, which were required to finish the first instruction, and $n-1$ cycles, which were required to finish the rest. So, the total number of cycles is $k + (n-1)$, assuming that there are no hazards, no stalls and perfectly balanced pipeline stages, hence the minimum number of cycles required to execute n instructions on a k -stage pipeline is $n+k-1$ cycles.